LAB SESSION 03

Doubly Linked List Operations

THEORY

A doubly linked list (DLL) is a type of linked data structure that consists of a set of sequentially linked records called nodes. Each node contains three fields:

1. **Data** – the value stored in the node.
2. **Next pointer** – a reference to the **next** node.
3. **Previous pointer** – a reference to the **previous** node.

This bidirectional structure allows **traversal in both directions**, making certain operations more efficient compared to a singly linked list.

Structure of a Node:

```
struct Node {
   int data;
   Node* prev;
   Node* next;
};
```

Key Operations:

1. **Traversal (Forward and Backward)**
2. **Insertion**
   - At the beginning
   - At the end
   - At a given position
3. **Deletion**
   - From the beginning
   - From the end
   - From a specific position

Table 3.1 Comparison with Singly Linked List

Feature	Singly Linked List	Doubly Linked List
Memory per node	2 fields	3 fields
Backward traversal	No	Yes
Insertion/deletion	Requires extra steps	Easier (if previous pointer available)
Performance	Less flexible	More flexible

PROCEDURE

1. Define a Node structure with data, prev, and next.
2. Create a DLL by allocating multiple nodes and linking them in both directions.
3. Implement forward and backward traversals.

4. Compile and run the code to verify correctness.

CODE

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* prev;
    Node* next;
};

void forwardPrint(Node* head) {
    Node* temp = head;
    cout << "Forward: ";
    while (temp != NULL) {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

void backwardPrint(Node* tail) {
    Node* temp = tail;
    cout << "Backward: ";
    while (temp != NULL) {
        cout << temp->data << " <-> ";
        temp = temp->prev;
    }
    cout << "NULL" << endl;
}

int main() {
    // Nodes creation
    Node* head = new Node();
    Node* second = new Node();
    Node* third = new Node();

    head->data = 10;
    head->prev = NULL;
    head->next = second;

    second->data = 20;
    second->prev = head;
    second->next = third;

    third->data = 30;
```

```
    third->prev = second;
    third->next = NULL;

    forwardPrint(head);
    backwardPrint(third);

    return 0;
}
```

EXPECTED OUTPUT

Forward: 10 <-> 20 <-> 30 <-> NULL
Backward: 30 <-> 20 <-> 10 <-> NULL

EXERCISE

1. Implement a function to insert a new node at the end.
2. Add a search function to find a given value.
3. Create a menu-driven program to test all operations dynamically.